

Unit 3: CSS (4hrs.)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

3.1 Introduction to CSS and CSS3

3.2 CSS Syntax, Selector and types (Internal, External, Inline)

3.3 CSS Properties

3.4. Box Model

3.5. Flexbox and Grid Layouts

3.6. Responsive Design Principles

Introduction to CSS

- CSS stands for Cascading Style Sheets.
- It is primarily used to provide styling to web pages, including color, layout, background, font, and border properties.
- The main objective of CSS is to improve content accessibility, provide enhanced flexibility and control, and specify presentation characteristics.

Introduction to CSS...

- CSS3 stands for Cascading Style Sheets Level 3.
- It is an advanced version of CSS, used for structuring, styling, and formatting web pages.
- CSS3 introduces several new features and is supported by all modern web browsers.
- One of the most significant advancements in CSS3 is the splitting of CSS standards into separate modules, making it simpler to learn and use.

New Features of CSS3

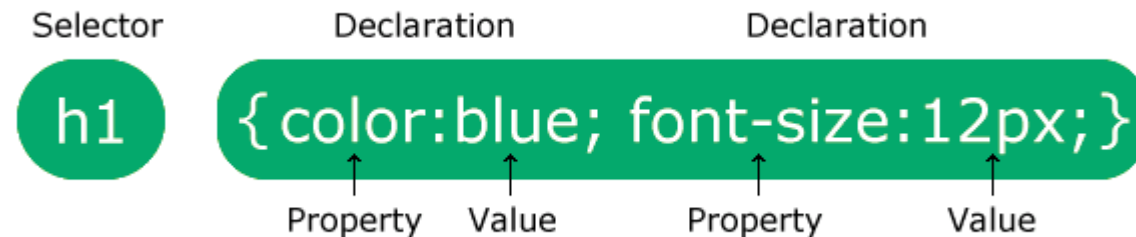
- **Combinator:** CSS3 introduces a new general sibling combinator, which matches sibling elements using the tilde (~) combinator.
- **CSS Selectors:** CSS3 selectors are more advanced than the simple selectors offered by CSS, providing a sequence of easy-to-use and simple selectors.
- **Pseudo-elements:** CSS3 adds many new pseudo-elements for easier and more detailed styling. The new convention of double colons (::) is also introduced.

New Features of CSS3

- Border Style: CSS3 includes new border styling features like border-radius, border-image-slice, border-image-source, and values for “width stretch”.
- Background Style Properties: CSS3 introduces new background style properties such as background-clip, background-size, background-style, and background-origin.

CSS Syntax

- CSS uses a set of syntax rules to define the styling of web pages.
- These rules consist of a selector and a declaration block.
- The selector is used to select the HTML element that you want to style, while the declaration block contains the properties and values that you want to apply to that element.
- Here's is the syntax:



Example

```
h1 {  
  font-size: 24px;  
  color: #333;  
}
```

- Here, h1 is the selector, and the declaration block contains the font-size and color properties with their respective values.

CSS Properties

- Some of the most popularly used CSS properties are color, font-family, font-size, background-color, margin, padding, border, and text-align.
- These properties are fundamental for controlling the visual appearance of web pages, including text styling, element spacing, and border design.
- Here's a more detailed look at each:
 - **color:** Sets the text color of an element.
 - **font-family:** Defines the font family to be used for the text.

CSS Properties

- **font-size**: Controls the size of the text.
- **background-color**: Sets the background color of an element.
- **margin**: Defines the space between an element and its adjacent elements.
- **padding**: Defines the space between an element's content and its border.
- **border**: Creates a border around an element.
- **text-align**: Controls the horizontal alignment of text within an element.
- **display**: sets the display type of an element

CSS Selectors

- CSS selectors are used to select the HTML elements that you want to style.
- There are several types of selectors:
 - **Element selector**: selects all elements of a specific type (e.g. h1)
 - **Class selector**: selects elements with a specific class name (e.g. .my-class)
 - **ID selector**: selects elements with a specific ID (e.g. #my-id)
 - **Attribute selector**: selects elements with a specific attribute (e.g. [data-attribute])
 - **Descendant selector**: selects elements that are descendants of another element (e.g. ul li)

Types of CSS

- CSS (Cascading Style Sheets) can be implemented in three primary ways:
 - inline,
 - internal, and
 - external.
- Inline CSS applies styles directly to individual HTML elements using the style attribute.
- Internal CSS defines styles within the <head> section of an HTML document using a <style> tag.

Types of CSS

- External CSS utilizes separate CSS files that are linked to one or more HTML documents.
- Here's a more detailed breakdown:
- **Inline CSS:** This method applies styles directly to an HTML element using the style attribute within the element's tag.
- For example:

```
<p style="color: blue; font-size: 16px;">This is some text.</p>
```

- While convenient for quick changes, inline CSS is generally not recommended for larger projects due to its lack of reusability and potential for clutter.

Types of CSS

- **Internal CSS:** This approach involves placing CSS rules within a <style> tag located in the <head> section of an HTML document.
- This allows you to style the entire document or specific elements within it, offering more organization than inline styles.

- For example:

```
<!DOCTYPE html>
<html>
<head>
<title>My Page</title>
<style>
p {
  color: green;
  font-size: 18px;
}
```

Types of CSS

```
h1 {  
  color: blue;  
}  
</style>  
</head>  
<body>  
  <h1>My Heading</h1>  
  <p>This is some text.</p>  
</body>  
</html>
```

Types of CSS

- **External CSS:** This is the preferred method for styling large web pages or projects.
- It involves creating a separate CSS file (e.g., styles.css) containing all the CSS rules and linking it to the HTML document using the <link> tag in the <head> section.
- This promotes code organization, reusability, and allows for easy maintenance and updates.

Types of CSS

For example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
    <link rel="stylesheet" href="styles.css" type="text/css" />
  </head>
  <body>
    <h1>My Heading</h1>
    <p>This is some text.</p>
  </body>
</html>
```

Types of CSS

- And the styles.css file would contain the following:

```
p {  
    color: green;  
    font-size: 18px;  
}  
h1 {  
    color: blue;  
}
```

CSS Frameworks

- CSS frameworks are pre-written CSS code that developers can use to style their websites.
- They typically include styles for common elements like buttons, forms, and navigation menus, making it faster and easier to create a website.
- Some popular CSS frameworks include Bootstrap, Foundation, and Materialize.
- Other CSS Frameworks like:
 - Tailwind CSS
 - Bulma

CSS Frameworks

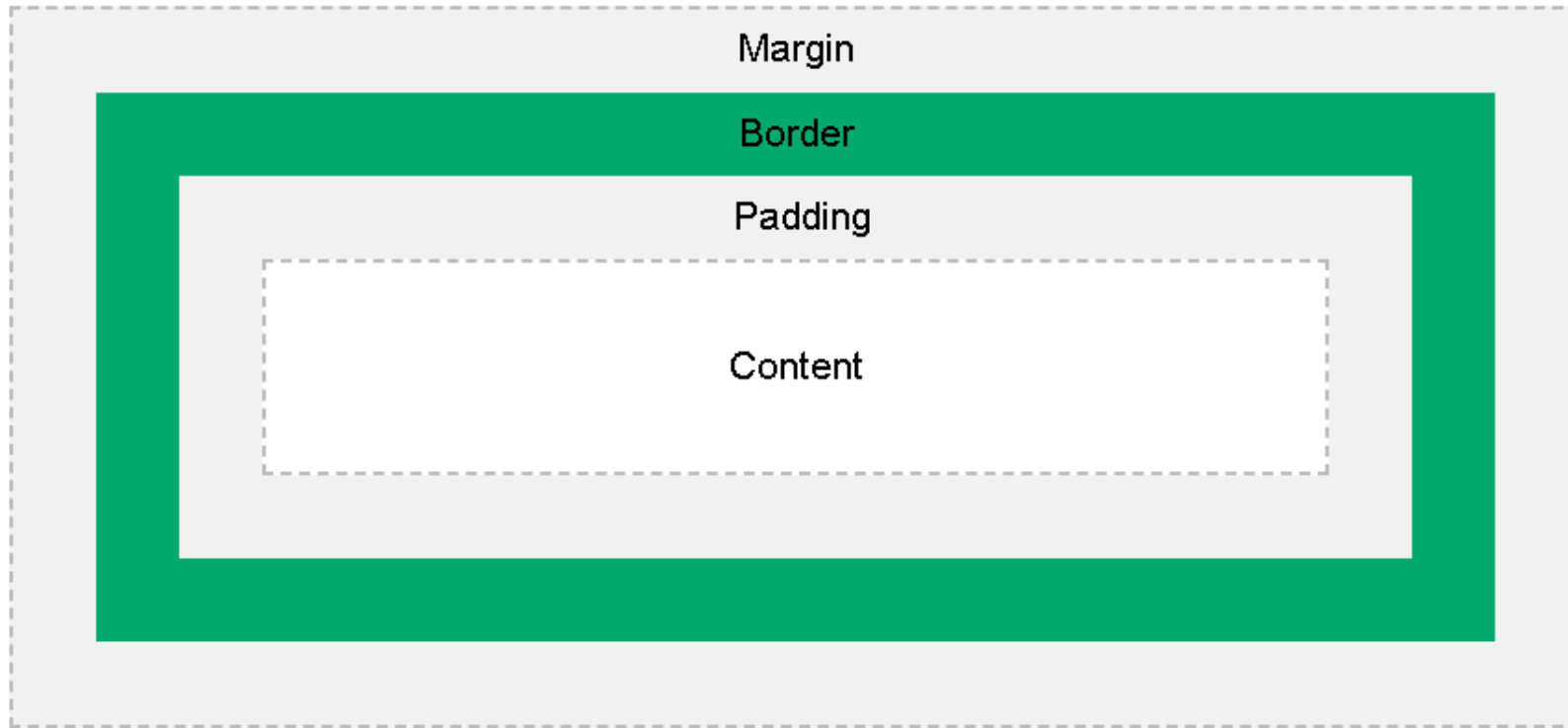
- UIKit
- Semantic UI
- Tachyons
- Milligram
- Fomantic-UI
- Primer
- Basscss
- Chota
- Open Props
- Vanilla framework
- Ant Design
- Luxa css
- Pico minimal css framework

CSS Box Model

- In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element.
- It consists of:
 - content,
 - padding,
 - borders,
 - and the margins.

CSS Box Model

- The image below illustrates the box model:



CSS Box Model

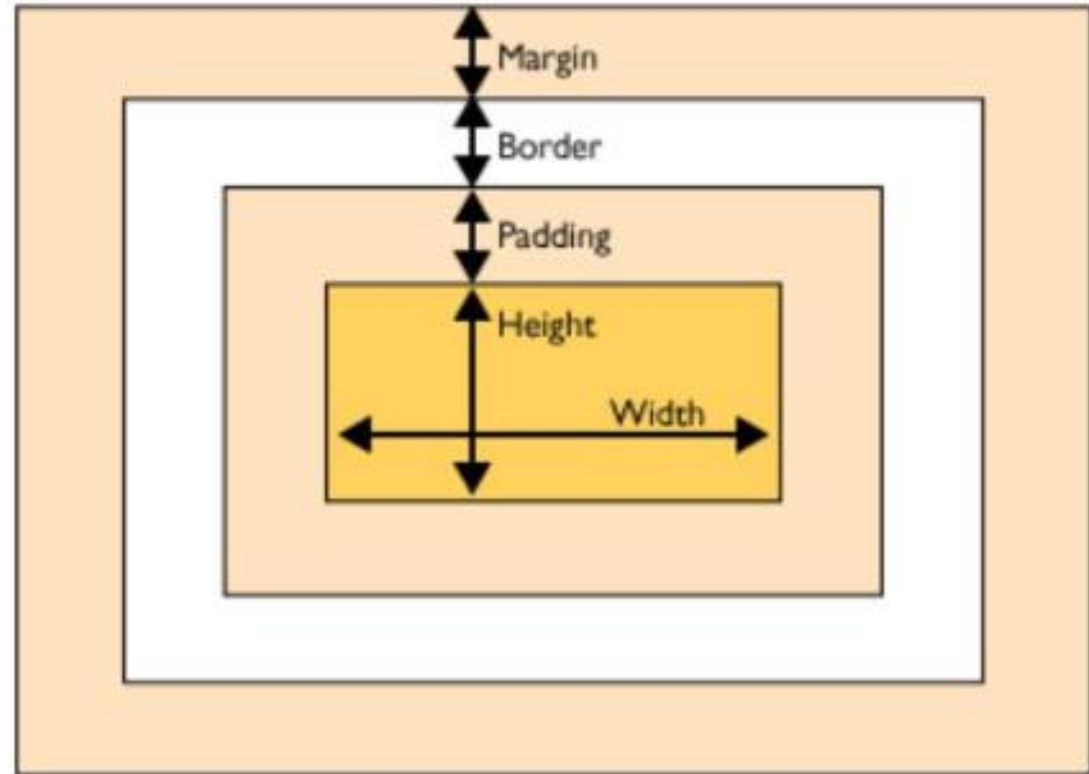
Explanation of the different parts:

- **Content:** - The content of the box, where text and images appear
- **Padding:** - Clears an area around the content. The padding is transparent
- **Border:** - A border that goes around the padding and content
- **Margin:** - Clears an area outside the border. The margin is transparent
- The box model allows us to add a border around elements, and to define space between elements.

CSS Box Model: An Example

- Demonstration of the box model:

```
div {  
  width: 300px;  
  height: 200px;  
  border: 15px solid green;  
  padding: 50px;  
  margin: 20px;  
}
```



CSS Box Model: An Example

- This <div> element will have a total width of 350px and a total height of 80px:

```
div {  
  width: 320px;  
  height: 50px;  
  padding: 10px;  
  border: 5px solid gray;  
  margin: 0;  
}
```

CSS Box Model: An Example

- Here is the calculation:

Total Width:

320px (width of content area) + 20px (left padding + right padding) + 10px (left border + right border)
= 350px (total width)

Total Height:

50px (height of content area) + 20px (top padding + bottom padding) + 10px (top border + bottom border)
= 80px (total height)

CSS Box Model: An Example

- The total width of an element should be calculated like this:
 - **Total element width = width + left padding + right padding + left border + right border + left margin + right margin**
- The total height of an element should be calculated like this:
 - **Total element height = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin**

Flexbox

- Flexbox is short for the Flexible Box Layout module.
- Flexbox is a layout method for arranging items in rows or columns.
- Flexbox makes it easier to design a flexible responsive layout structure, without using float or positioning.

Flex Box Layout Module

- Before the Flexible Box Layout module, there were four layout modes:
 - Block, for sections in a webpage
 - Inline, for text
 - Table, for two-dimensional table data
 - Positioned, for explicit position of an element
- CSS flexbox is supported in all modern browsers.

Flexbox Components

- A flexbox always consists of:
 - a **Flex Container** - the parent (container) <div> element
 - **Flex Items** - the items inside the container <div>
- To start using CSS Flexbox, you need to first define a flex container.
- The flex container becomes flexible by setting the display property to flex.

CSS Flexbox Example

```
<style>
```

```
.flex-container {  
  display: flex;  
  background-color: #ccc;  
}
```

```
.flex-container > div {  
  background-color: #f1f1f1;
```

```
margin: 10px;  
padding: 20px;  
font-size: 30px;  
color: red;  
}
```

```
</style>
```

CSS Flexbox Example

```
<body>  
<div class="flex-container">  
  <div>BCA</div>  
  <div>BE</div>  
  <div>Computer</div>  
</div>  
</body>
```



Flex container properties

- flex-direction
- flex-wrap
- flex-flow
- justify-content
- align-items
- align-content

Flex container properties

Computer

BE

BCA

- The flex-direction property specifies the display-direction of flex items in the flex container.
- The flex-direction property can have one of the following values:
- flex-direction
 - row
 - column
 - row-reverse
 - column-reverse

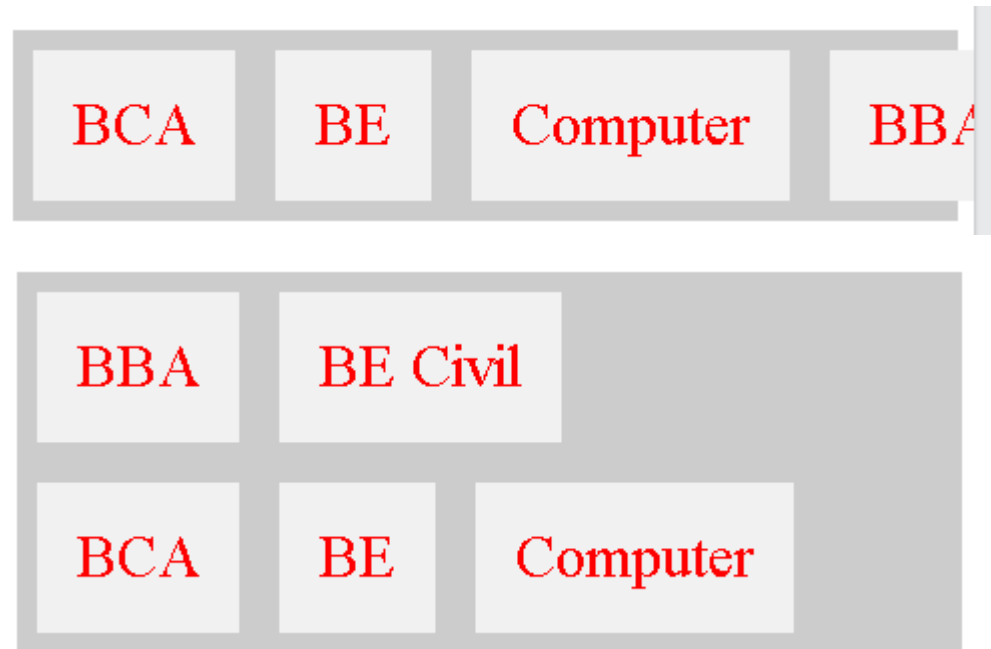
Computer

BE

BCA

Flex container properties

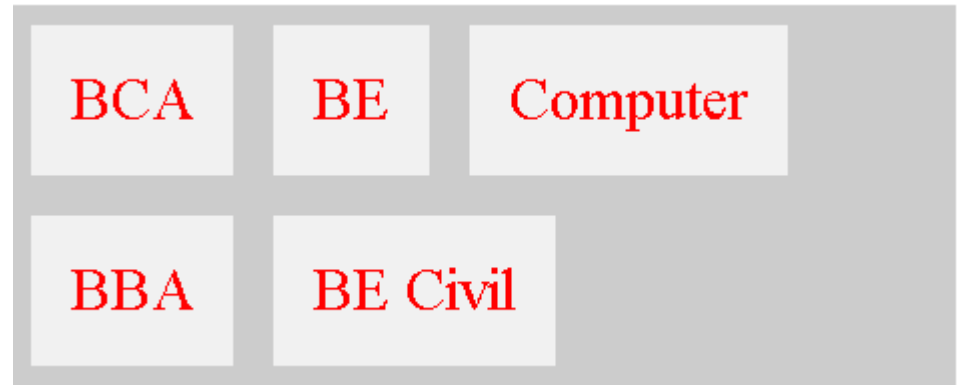
- The flex-wrap property specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.
- The flex-wrap property can have one of the following values:
- flex-wrap
 - nowrap
 - wrap
 - wrap-reverse



Flex container properties

- The flex-flow property is a shorthand property for setting both the flex-direction and flex-wrap properties.
- flex-flow

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap;  
}
```



Flex container properties

- The justify-content property is used to align the flex items when they do not use all available space on the main-axis (horizontally).
- The justify-content property can have one of the following values:
- justify-content
 - center
 - flex-start
 - flex-end
 - space-around
 - space-between
 - space-evenly

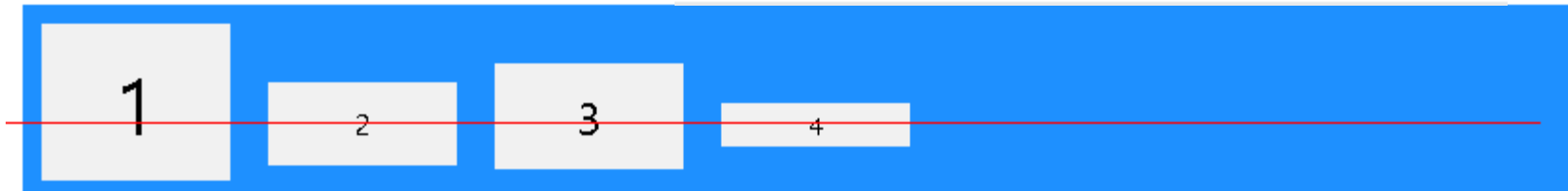
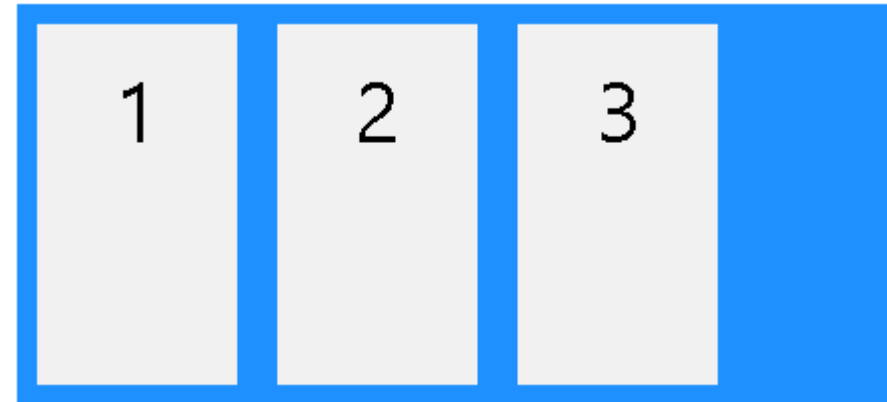
Flex container properties

- The align-items property is used to align the flex items when they do not use all available space on the cross-axis (vertically).
- The align-items property can have one of the following values:

- align-items

- center
- flex-start
- flex-end
- stretch
- baseline
- normal

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: stretch;  
}
```



Flex container properties

- The align-content property is used to align the flex lines.
- The align-content property is similar to align-items, but instead of aligning flex items, it aligns the flex lines.
- The align-content property can have one of the following values:
- justify-content
 - center
 - flex-start
 - flex-end
 - space-around
 - space-between
 - space-evenly

Flex Items

- The direct child elements of a flex container automatically becomes flex items.



- The element above represents four blue flex items inside a grey flex container.

Flex Item properties

- order
- flex-grow
- flex-shrink
- flex-basis
- flex
- align-self

Flex Item properties

- The **order** property specifies the order of the flex items inside the flex container.
- The first flex item in the code does not have to appear as the first item in the layout.
- The order value must be a number, default value is 0.

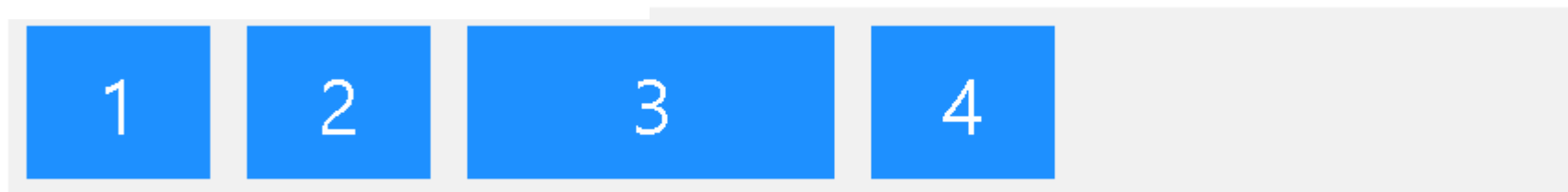
```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```



Flex Item properties

- The flex-shrink property specifies how much a flex item will shrink relative to the rest of the flex items.
- The value must be a number, default value is 1.
- The flex-basis property specifies the initial length of a flex item.

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="flex-basis: 200px">3</div>  
  <div>4</div>  
</div>
```



Flex Item properties

- The flex property is a shorthand property for the flex-grow, flex-shrink, and flex-basis properties.

Grid Layout

- The Grid Layout Module offers a grid-based layout system, with rows and columns.
- The Grid Layout Module allows developers to easily create complex web layouts.
- The Grid Layout Module makes it easier to design a responsive layout structure, without using float or positioning.
- The CSS grid properties are supported in all modern browsers.

Grid Components

- A grid always consists of:
 - Grid Container - the parent (container) `<div>` element
 - Grid Items - the items inside the container `<div>`
- A grid layout consists of a parent element (the grid container), with one or more grid items.
- All direct children of the grid container automatically become grid items.

Grid Example

- The <div> element becomes a grid container when its display property is set to grid or inline-grid.

```
<style>
```

```
.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  background-color: dodgerblue;  
  padding: 10px;  
}
```

```
.container > div {  
  background-color: #f1f1f1;  
  border: 1px solid black;  
  padding: 20px;  
  font-size: 30px;  
  text-align: center;  
}  
</style>
```

Grid Example

1	2	3
4	5	6
7	8	

1	2	3
4	5	6
7	8	

Grid Properties

- The vertical lines of grid items are called *columns*.
- The horizontal lines of grid items are called *rows*.
- The spaces between each column/row are called *gaps*.
- You can adjust the gap size by using one of the following properties:
 - column-gap
 - row-gap
 - gap
- The column-gap property specifies the gap between the columns in a grid.
- The row-gap property specifies the gap between the rows in a grid.

Grid Properties

- The gap property is a shorthand property for row-gap and column-gap
- The lines between columns are called column lines.
- The lines between rows are called row lines.
- We can specify where to start and end a grid item by using the following properties:
 - grid-column-start
 - grid-column-end
 - grid-row-start
 - grid-row-end
 - grid-column
 - grid-row
- You can refer to line numbers when placing a grid item in a grid container.

Grid Example

```
.grid-container {  
  display: grid;  
  grid-template-columns: 80px 200px auto 40px;  
}
```

1	2	3	4
5	6	7	8

Grid Items

- A grid container contains one or more grid items.
- By default, a container has one grid item for each column, in each row, but you can style the grid items so that they will span multiple columns and/or rows.

```
.item1 {  
  grid-column: 1 / span 2;  
}
```

1		2
3	4	5
6	7	8

Grid Items

- Make "item2" start on column 2 and span 2 columns

```
.item2 {  
  grid-column: 2 / span 2;  
}
```

1	2	
3	4	5
6	7	8

Grid Items

- Place the first grid item at row line 1, and let it end on row line 3

```
.item1 {  
  grid-row-start: 1;  
  grid-row-end: 3;  
}
```

1	2	3
	4	5
6	7	8

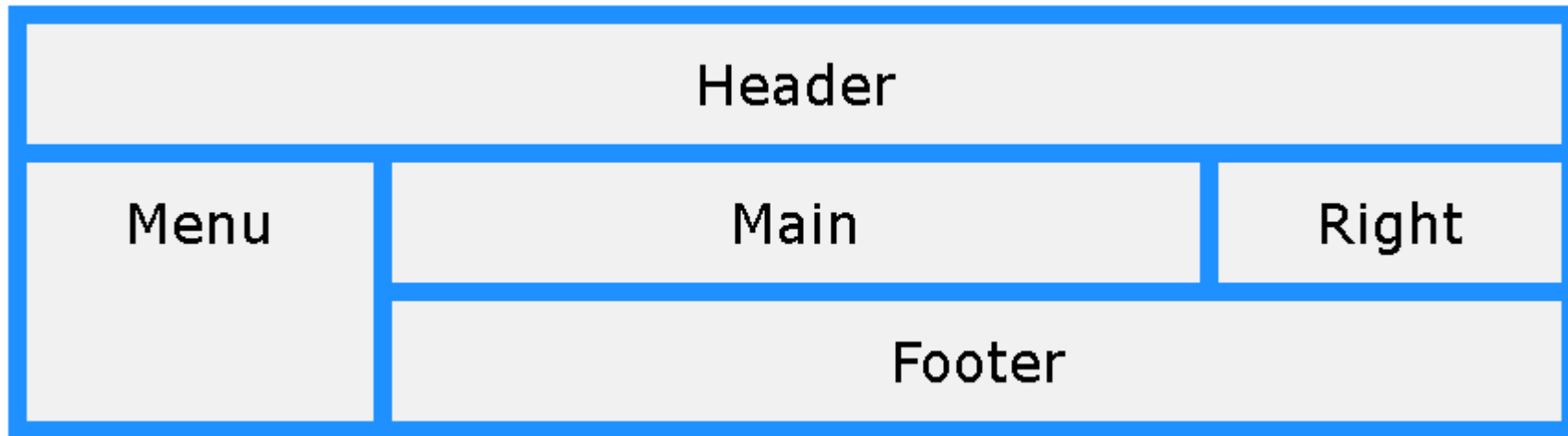
Example

```
<style>
.grid-container {
  display: grid;
  grid-template-areas:
    'header header header header header
header'
    'menu main main main main right'
    'menu footer footer footer footer footer';
  gap: 10px;
  background-color: dodgerblue;
  padding: 10px;
}
.grid-container > div {
```

```
  background-color: #f1f1f1;
  color: #000;
  padding: 10px;
  font-size: 30px;
  text-align: center;
}
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }
</style>
```

Example

```
<div class="grid-container">  
  <div class="item1">Header</div>  
  <div class="item2">Menu</div>  
  <div class="item3">Main</div>  
  <div class="item4">Right</div>  
  <div class="item5">Footer</div>  
</div>
```

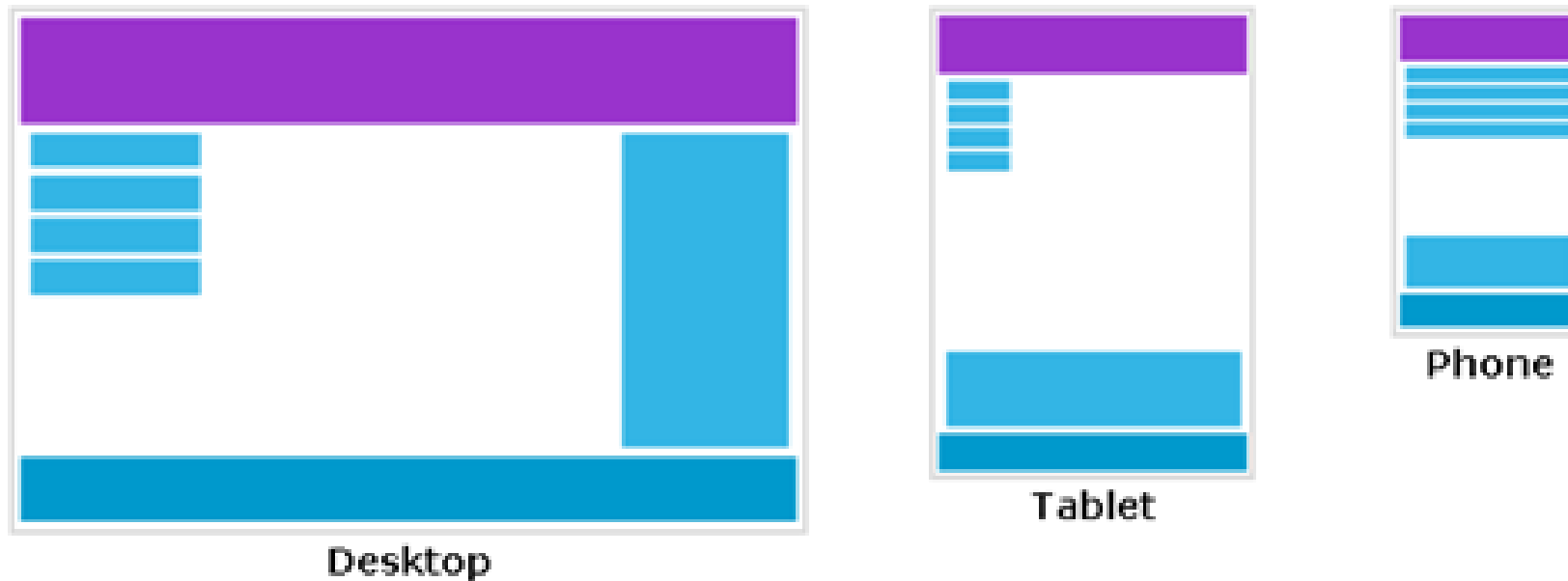


Responsive Design

- Responsive web design makes your web page look good on all devices.
- Responsive web design uses only HTML and CSS.
- Responsive web design is not a program or a JavaScript.
- Web pages can be viewed using many different devices: desktops, tablets, and phones.
- Your web page should look good, and be easy to use, regardless of the device.

Responsive Design

- Web pages should not leave out information to fit smaller devices, but rather adapt its content to fit any device:



Responsive Design Principals

- Responsive design ensures websites are accessible and enjoyable across various devices and screen sizes.
- Key principles include using fluid grids and images, utilizing media queries to adjust layouts based on device characteristics, and focusing on mobile-first design, ensuring a seamless user experience.
- The Interaction Design Foundation notes that these principles aim to create a consistent and user-friendly experience on all devices.

Responsive Design Principals

- Here's a more detailed look at the core principles:

1. Fluid Grids
2. Fluid Images
3. Media Queries
4. Mobile-First Approach
5. Breakpoints
6. Viewport Configuration
7. Testing on Real Devices

Chapter Concluded

Next Chapter....